

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Простые методы сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Метод вставки

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

1. Исходные данные: массив a размером n .
2. Массив делится на две логические части:
 - отсортированная — элементы $a[0 \dots i-1]$;
 - неотсортированная — элементы $a[i \dots n-1]$.

Изначально отсортированная часть содержит один элемент $a[0]$, граница находится после него: $a[] = \{a[0] \mid a[1], \dots, a[n-1]\}$.

3. Внешний цикл `for (i = 1; i < n; i++)` перебирает элементы неотсортированной части, начиная с индекса 1:

а) Сохраняем текущий элемент в переменную: $temp = a[i]$.

б) Устанавливаем индекс $j = i - 1$ — последний элемент отсортированной части.

4. Внутренний цикл `while (j >= 0 && a[j] > temp)` выполняет поиск позиции для вставки:

а) Сравниваем $temp$ с элементами отсортированной части справа налево.

б) Пока $a[j] > temp$, сдвигаем элемент вправо: $a[j + 1] = a[j]$, затем $j--$.

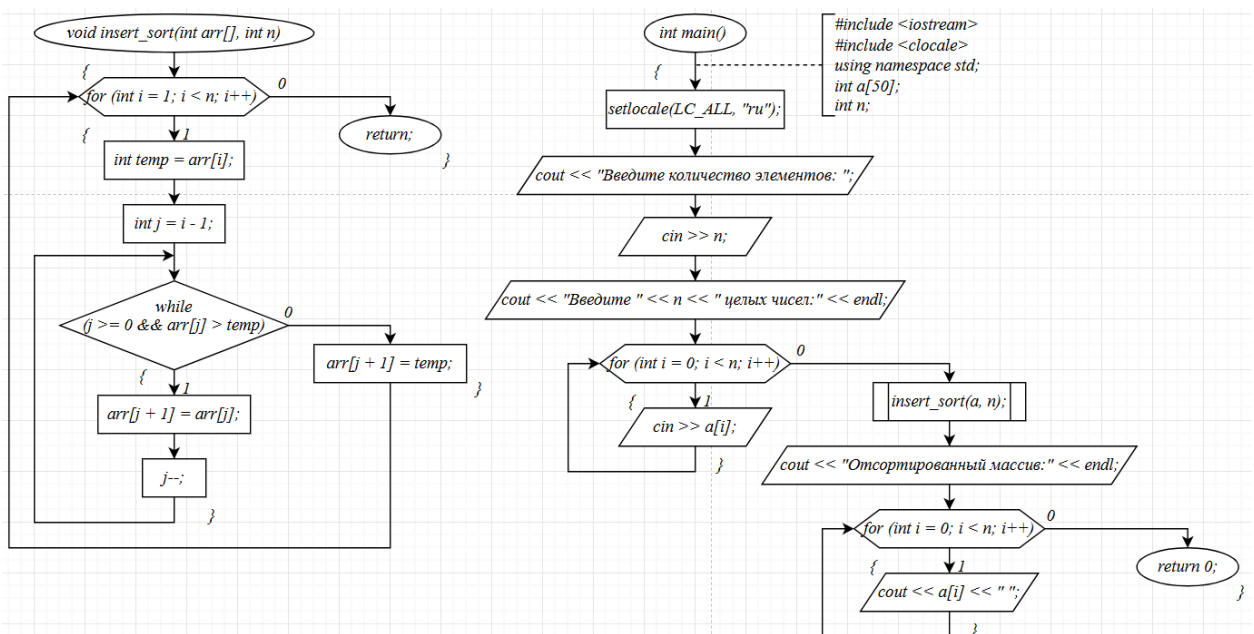
в) Цикл останавливается, когда найден элемент $a[j] \leq temp$ или достигнуто начало массива ($j = -1$).

5. Вставляем сохранённый элемент на освободившееся место: $a[j + 1] = temp$.

6. Граница отсортированной части автоматически расширяется на один элемент (переход к следующей итерации внешнего цикла).

7. После завершения внешнего цикла ($i = n$) массив полностью отсортирован по возрастанию.

3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <locale>
3.
4. using namespace std;
5.
6. void printArray(int arr[], int n, int border) {
7.     cout << "{";
8.     for (int i = 0; i < n; i++) {
9.         cout << arr[i];
10.        if (i == border - 1) cout << "|";
11.        if (i < n - 1) cout << ", ";
12.    }
13.    cout << "}" << endl;
14.}
15.
16. void insert_sort(int arr[], int n) {
17.    printArray(arr, n, 1);
18.
19.    for (int i = 1; i < n; i++) {
20.        int temp = arr[i];
21.        int j = i - 1;
22.
23.        cout << "\nШаг " << i << ": temp = " << temp << endl;
24.
25.        while (j >= 0 && arr[j] > temp) {
26.            cout << arr[j] << " > " << temp << " - да, сдвигаем" << endl;
27.            arr[j + 1] = arr[j];
28.            j--;
29.        }
30.
31.        if (j >= 0) {
32.            cout << arr[j] << " > " << temp << " - нет" << endl;
33.        }
34.
35.        arr[j + 1] = temp;
36.        printArray(arr, n, i + 1);
37.    }
38.}
39.
40. int main() {
41.    setlocale(LC_ALL, "ru");
42.    int a[50];
43.    int n;
44.
45.    cout << "Введите количество элементов: ";
46.    cin >> n;
47.
48.    cout << "Введите " << n << " целых чисел:" << endl;
49.    for (int i = 0; i < n; i++) {
50.        cin >> a[i];
51.    }
```

```
52.  
53.  insert_sort(a, n);  
54.  
55.  cout << "\nОтсортированный массив: ";  
56.  for (int i = 0; i < n; i++) {  
57.      cout << a[i] << " ";  
58.  }  
59.  cout << endl;  
60.  
61.  return 0;  
62. }
```

5. Результат выполнения программы

```
Введите количество элементов: 10
Введите 10 целых чисел:
45
8
38
92
15
48
29
47
2
65
{45|, 8, 38, 92, 15, 48, 29, 47, 2, 65}

Шаг 1: temp = 8
45 > 8 - да, сдвигаем
{8, 45|, 38, 92, 15, 48, 29, 47, 2, 65}

Шаг 2: temp = 38
45 > 38 - да, сдвигаем
8 > 38 - нет
{8, 38, 45|, 92, 15, 48, 29, 47, 2, 65}

Шаг 3: temp = 92
45 > 92 - нет
{8, 38, 45, 92|, 15, 48, 29, 47, 2, 65}

Шаг 4: temp = 15
92 > 15 - да, сдвигаем
45 > 15 - да, сдвигаем
38 > 15 - да, сдвигаем
8 > 15 - нет
{8, 15, 38, 45, 92|, 48, 29, 47, 2, 65}

Шаг 5: temp = 48
92 > 48 - да, сдвигаем
45 > 48 - нет
{8, 15, 38, 45, 48, 92|, 29, 47, 2, 65}
```

```
Шаг 6: temp = 29
92 > 29 - да, сдвигаем
48 > 29 - да, сдвигаем
45 > 29 - да, сдвигаем
38 > 29 - да, сдвигаем
15 > 29 - нет
{8, 15, 29, 38, 45, 48, 92|, 47, 2, 65}

Шаг 7: temp = 47
92 > 47 - да, сдвигаем
48 > 47 - да, сдвигаем
45 > 47 - нет
{8, 15, 29, 38, 45, 47, 48, 92|, 2, 65}

Шаг 8: temp = 2
92 > 2 - да, сдвигаем
48 > 2 - да, сдвигаем
47 > 2 - да, сдвигаем
45 > 2 - да, сдвигаем
38 > 2 - да, сдвигаем
29 > 2 - да, сдвигаем
15 > 2 - да, сдвигаем
8 > 2 - да, сдвигаем
{2, 8, 15, 29, 38, 45, 47, 48, 92|, 65}

Шаг 9: temp = 65
92 > 65 - да, сдвигаем
48 > 65 - нет
{2, 8, 15, 29, 38, 45, 47, 48, 65, 92|}

Отсортированный массив: 2 8 15 29 38 45 47 48 65 92
```

II. Метод выбора

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

1. Исходные данные: массив a размером n .

2. Массив делится на две логические части:

- отсортированная — элементы $a[0 \dots i-1]$;
- неотсортированная — элементы $a[i \dots n-1]$.

Изначально отсортированная часть пуста, граница находится перед первым элементом.

3. Внешний цикл $\text{for } (i = 0; i < n-1; i++)$ расширяет отсортированную часть:

а) Устанавливаем индекс минимального элемента: $\text{min} = i$ (предполагаем, что первый элемент неотсортированной части — минимальный).

б) Внутренний цикл $\text{for } (j = i+1; j < n; j++)$ перебирает оставшуюся неотсортированную часть:

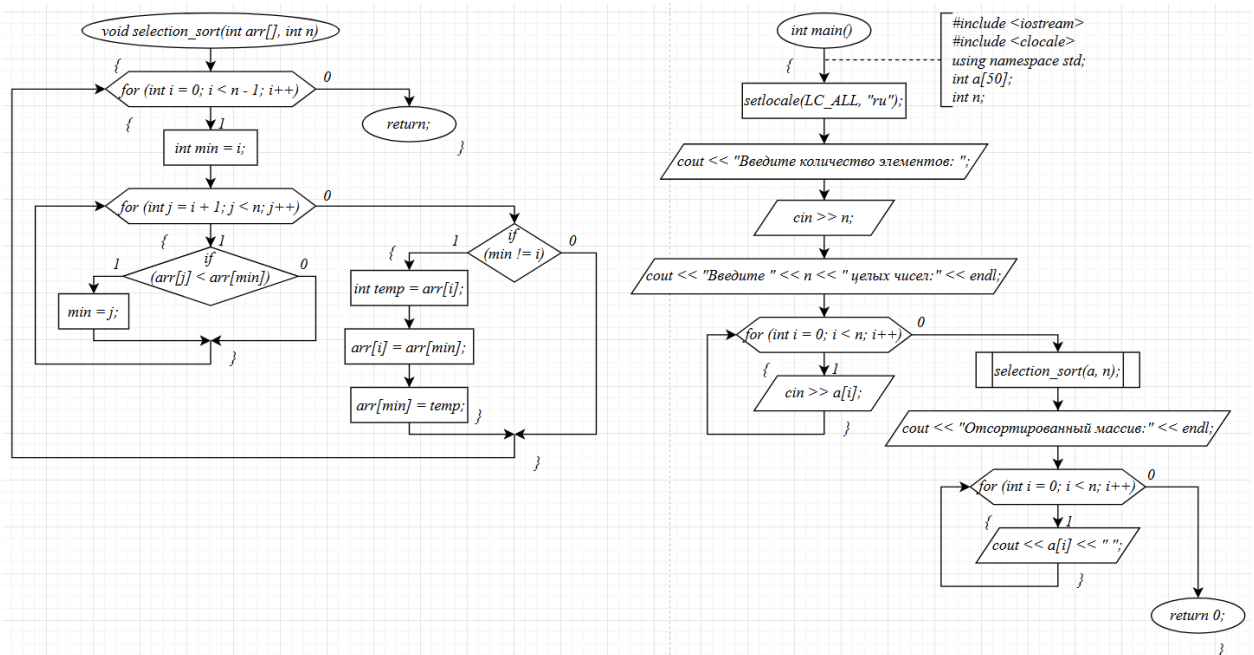
- если $a[j] < a[\text{min}]$, обновляем индекс: $\text{min} = j$.

в) После завершения внутреннего цикла меняем местами $a[i]$ и $a[\text{min}]$ (если $\text{min} \neq i$), тем самым помещая минимальный элемент в конец отсортированной части.

4. Граница отсортированной части автоматически сдвигается вправо на один элемент после каждой итерации внешнего цикла.

5. После $n-1$ итераций внешнего цикла массив полностью отсортирован по возрастанию (последний элемент автоматически занимает своё место).

3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <locale>
3.
4. using namespace std;
5.
6. void printArray(int arr[], int n, int border) {
7.     cout << "a[] = {";
8.     for (int i = 0; i < n; i++) {
9.         cout << arr[i];
10.        if (i == border - 1) cout << "|";
11.        if (i < n - 1) cout << ", ";
12.    }
13.    cout << "}" << endl;
14.}
15.
16. void selection_sort(int arr[], int n) {
17.    printArray(arr, n, 0);
18.
19.    for (int i = 0; i < n - 1; i++) {
20.        int min = i;
21.
22.        for (int j = i + 1; j < n; j++) {
23.            if (arr[j] < arr[min]) {
24.                min = j;
25.            }
26.        }
27.
28.        cout << "\nШаг " << i << ": min = a[" << min << "] = " << arr[min] << endl;
29.
30.        if (min != i) {
31.            cout << "Меняем a[" << i << "] u a[" << min << "]" << endl;
32.            int temp = arr[i];
33.            arr[i] = arr[min];
34.            arr[min] = temp;
35.        }
36.        else {
37.            cout << "Без обмена" << endl;
38.        }
39.
40.        printArray(arr, n, i + 1);
41.    }
42.}
43.
44. int main() {
```

```
45.  setlocale(LC_ALL, "ru");
46.  int a[50];
47.  int n;
48.
49.  cout << "Введите количество элементов: ";
50.  cin >> n;
51.
52.  cout << "Введите " << n << " целых чисел:" << endl;
53.  for (int i = 0; i < n; i++) {
54.      cin >> a[i];
55.  }
56.
57.  selection_sort(a, n);
58.
59.  cout << "\nОтсортированный массив:" << endl;
60.  for (int i = 0; i < n; i++) {
61.      cout << a[i] << " ";
62.  }
63.  cout << endl;
64.
65.  return 0;
66. }
```


5. Результат выполнения программы

```
Введите количество элементов: 10
Введите 10 целых чисел:
45
978
69
10
39
46
16
56
29
82
a[] = {45, 978, 69, 10, 39, 46, 16, 56, 29, 82}

Шаг 0: min = a[3] = 10
Меняем a[0] и a[3]
a[] = {10, 978, 69, 45, 39, 46, 16, 56, 29, 82}

Шаг 1: min = a[6] = 16
Меняем a[1] и a[6]
a[] = {10, 16, 69, 45, 39, 46, 978, 56, 29, 82}

Шаг 2: min = a[8] = 29
Меняем a[2] и a[8]
a[] = {10, 16, 29, 45, 39, 46, 978, 56, 69, 82}

Шаг 3: min = a[4] = 39
Меняем a[3] и a[4]
a[] = {10, 16, 29, 39, 45, 46, 978, 56, 69, 82}

Шаг 4: min = a[4] = 45
Без обмена
a[] = {10, 16, 29, 39, 45, 46, 978, 56, 69, 82}

Шаг 5: min = a[5] = 46
Без обмена
a[] = {10, 16, 29, 39, 45, 46, 978, 56, 69, 82}

Шаг 6: min = a[7] = 56
Меняем a[6] и a[7]
a[] = {10, 16, 29, 39, 45, 46, 56, 978, 69, 82}

Шаг 7: min = a[8] = 69
Меняем a[7] и a[8]
a[] = {10, 16, 29, 39, 45, 46, 56, 69, 978, 82}

Шаг 8: min = a[9] = 82
Меняем a[8] и a[9]
a[] = {10, 16, 29, 39, 45, 46, 56, 69, 82, 978}

Отсортированный массив:
10 16 29 39 45 46 56 69 82 978
```

III. Метод обмена (пузырька)

1. Постановка задачи

Отсортировать массив из n элементов по возрастанию.

2. Анализ задачи

1. Исходные данные: массив a размером n .

2. Массив логически делится на две части:

- неотсортированная — элементы $a[0 \dots n-i-1]$;

- отсортированная — элементы $a[n-i \dots n-1]$ (накапливается справа).

Изначально отсортированная часть пуста.

3. Внешний цикл $\text{for } (i = 0; i < n-1; i++)$ определяет количество проходов по массиву (максимум $n-1$ проходов):

а) Внутренний цикл $\text{for } (j = 0; j < n-i-1; j++)$ последовательно сравнивает соседние элементы в неотсортированной части:

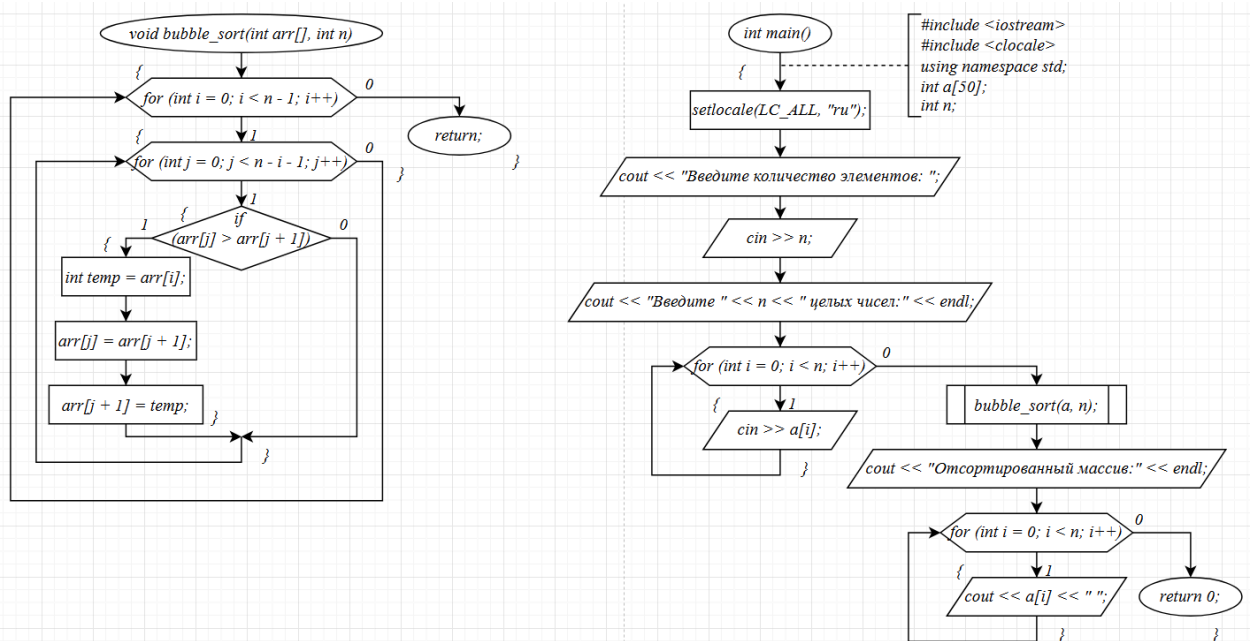
- если $a[j] > a[j+1]$, выполняем обмен: $\text{temp} = a[j]$; $a[j] = a[j+1]$; $a[j+1] = \text{temp}$.

б) После каждого прохода внутреннего цикла максимальный элемент неотсортированной части «всплывает» на правую границу и присоединяется к отсортированной части.

4. Граница отсортированной части сдвигается влево на один элемент после каждой итерации внешнего цикла (учтено в условии $j < n-i-1$).

5. После завершения внешнего цикла массив полностью отсортирован по возрастанию.

3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <locale>
3.
4. using namespace std;
5.
6. // Функция вывода массива с границей '|'
7. void printArray(int arr[], int n, int sorted) {
8.     cout << "a[] = {";
9.     for (int i = 0; i < n; i++) {
10.         cout << arr[i];
11.         if (i == n - sorted - 1) cout << "|";
12.         if (i < n - 1) cout << ", ";
13.     }
14.     if (sorted == n) cout << "|";
15.     cout << "}" << endl;
16. }
17.
18. void bubble_sort(int arr[], int n) {
19.     printArray(arr, n, 0);
20.
21.     for (int i = 0; i < n - 1; i++) {
22.         cout << "\n--- Проход " << i + 1 << " ---" << endl;
23.         int swaps = 0;
24.
25.         for (int j = 0; j < n - i - 1; j++) {
26.             cout << "Сравниваем " << arr[j] << " u " << arr[j + 1] << " - ";
27.
28.             if (arr[j] > arr[j + 1]) {
29.                 cout << "меняем" << endl;
30.                 int temp = arr[j];
31.                 arr[j] = arr[j + 1];
32.                 arr[j + 1] = temp;
33.                 swaps++;
34.                 printArray(arr, n, i + 1);
35.             }
36.             else {
37.                 cout << "нет" << endl;
38.             }
39.         }
40.
41.         if (swaps == 0) {
42.             cout << "Больше обменов не нужно, завершаем" << endl;
43.             break;
44.         }
45.     }
46. }
47.
48. int main() {
49.     setlocale(LC_ALL, "ru");
50.     int a[50];
51.     int n;
52.
53.     cout << "Введите количество элементов: ";
```

```
54.  cin >> n;
55.
56.  cout << "Введите " << n << " целых чисел:" << endl;
57.  for (int i = 0; i < n; i++) {
58.      cin >> a[i];
59.  }
60.
61.  bubble_sort(a, n);
62.
63.  cout << "\nОтсортированный массив:" << endl;
64.  for (int i = 0; i < n; i++) {
65.      cout << a[i] << " ";
66.  }
67.  cout << endl;
68.
69.  return 0;
70. }
```

5. Результат выполнения программы

```
Введите количество элементов: 10
Введите 10 целых чисел:
56
80
26
47
18
58
12
45
67
98
a[] = {56, 80, 26, 47, 18, 58, 12, 45, 67, 98|}

--- Проход 1 ---
Сравниваем 56 и 80 – нет
Сравниваем 80 и 26 – меняем
a[] = {56, 26, 80, 47, 18, 58, 12, 45, 67|, 98}
Сравниваем 80 и 47 – меняем
a[] = {56, 26, 47, 80, 18, 58, 12, 45, 67|, 98}
Сравниваем 80 и 18 – меняем
a[] = {56, 26, 47, 18, 80, 58, 12, 45, 67|, 98}
Сравниваем 80 и 58 – меняем
a[] = {56, 26, 47, 18, 58, 80, 12, 45, 67|, 98}
Сравниваем 80 и 12 – меняем
a[] = {56, 26, 47, 18, 58, 12, 80, 45, 67|, 98}
Сравниваем 80 и 45 – меняем
a[] = {56, 26, 47, 18, 58, 12, 45, 80, 67|, 98}
Сравниваем 80 и 67 – меняем
a[] = {56, 26, 47, 18, 58, 12, 45, 67, 80|, 98}
Сравниваем 80 и 98 – нет

--- Проход 2 ---
Сравниваем 56 и 26 – меняем
a[] = {26, 56, 47, 18, 58, 12, 45, 67|, 80, 98}
Сравниваем 56 и 47 – меняем
a[] = {26, 47, 56, 18, 58, 12, 45, 67|, 80, 98}
Сравниваем 56 и 18 – меняем
a[] = {26, 47, 18, 56, 58, 12, 45, 67|, 80, 98}
Сравниваем 56 и 58 – нет
Сравниваем 58 и 12 – меняем
a[] = {26, 47, 18, 56, 12, 58, 45, 67|, 80, 98}
Сравниваем 58 и 45 – меняем
a[] = {26, 47, 18, 56, 12, 45, 58, 67|, 80, 98}
Сравниваем 58 и 67 – нет
Сравниваем 67 и 80 – нет
```

```
--- Проход 3 ---
Сравниваем 26 и 47 – нет
Сравниваем 47 и 18 – меняем
a[] = {26, 18, 47, 56, 12, 45, 58|, 67, 80, 98}
Сравниваем 47 и 56 – нет
Сравниваем 56 и 12 – меняем
a[] = {26, 18, 47, 12, 56, 45, 58|, 67, 80, 98}
Сравниваем 56 и 45 – меняем
a[] = {26, 18, 47, 12, 45, 56, 58|, 67, 80, 98}
Сравниваем 56 и 58 – нет
Сравниваем 58 и 67 – нет

--- Проход 4 ---
Сравниваем 26 и 18 – меняем
a[] = {18, 26, 47, 12, 45, 56|, 58, 67, 80, 98}
Сравниваем 26 и 47 – нет
Сравниваем 47 и 12 – меняем
a[] = {18, 26, 12, 47, 45, 56|, 58, 67, 80, 98}
Сравниваем 47 и 45 – меняем
a[] = {18, 26, 12, 45, 47, 56|, 58, 67, 80, 98}
Сравниваем 47 и 56 – нет
Сравниваем 56 и 58 – нет

--- Проход 5 ---
Сравниваем 18 и 26 – нет
Сравниваем 26 и 12 – меняем
a[] = {18, 12, 26, 45, 47|, 56, 58, 67, 80, 98}
Сравниваем 26 и 45 – нет
Сравниваем 45 и 47 – нет
Сравниваем 47 и 56 – нет

--- Проход 6 ---
Сравниваем 18 и 12 – меняем
a[] = {12, 18, 26, 45|, 47, 56, 58, 67, 80, 98}
Сравниваем 18 и 26 – нет
Сравниваем 26 и 45 – нет
Сравниваем 45 и 47 – нет

--- Проход 7 ---
Сравниваем 12 и 18 – нет
Сравниваем 18 и 26 – нет
Сравниваем 26 и 45 – нет
Больше обменов не нужно, завершаем

Отсортированный массив:
12 18 26 45 47 56 58 67 80 98
```